[Process Synchronization]

Considered there are N philosophers seated around a circular table with one chopstick between each pair of philosophers. There is one chopstick between each philosopher. A philosopher may eat if he can pick up the two chopsticks adjacent to him. One chopstick may be picked up by any one of its adjacent followers but not both. Write a program to solve the problem using process synchronization technique.

Code:-

```
chanchal-manwatkar@chanchal-manwatkar-Aspire-A515-56G:~$ cat practical6.c
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define N 5
sem_t chopstick[N];
sem_t room;   // To prevent deadlock
void* philosopher(void* num) {
    int id = *(int*)num;
    printf("Philosopher %d is thinking\n", id);
    sleep(1);
    sem_wait(&room);  // Allow only N-1 philosophers
    sem_wait(&chopstick[id]);              // Pick left
    sem_wait(&chopstick[(id + 1) % N]);    // Pick right
    printf("Philosopher %d is eating\n", id);
    sleep(2);
    sem_post(&chopstick[id]);              // Put left
    sem_post(&chopstick[(id + 1) % N]);    // Put right
    sem_post(&room);   // Leave room
    printf("Philosopher %d finished eating\n", id);
    return NULL;
}
int main() {
    pthread_t ph[N];
    int id[N];

    sem_init(&room, 0, N-1);  // Only N-1 philosophers allowed

    for(int i = 0; i < N; i++)
        sem_init(&chopstick[i], 0, 1);

    for(int i = 0; i < N; i++) {
        id[i] = i;
        pthread_create(&ph[i], NULL, philosopher, &id[i]);
    }

    for(int i = 0; i < N; i++)
        pthread_join(ph[i], NULL);

    return 0;
}
chanchal-manwatkar@chanchal-manwatkar-Aspire-A515-56G:~$
```

Output:-

```
chanchal-manwatkar@chanchal-manwatkar-Aspire-A515-56G:~$ nano practical6.c
chanchal-manwatkar@chanchal-manwatkar-Aspire-A515-56G:~$ gcc practical6.c -o practical6
chanchal-manwatkar@chanchal-manwatkar-Aspire-A515-56G:~$
chanchal-manwatkar@chanchal-manwatkar-Aspire-A515-56G:~$ ./practical6
Philosopher 0 is thinking
Philosopher 1 is thinking
Philosopher 2 is thinking
Philosopher 3 is thinking
Philosopher 4 is thinking
Philosopher 3 is eating
Philosopher 1 is eating
Philosopher 3 finished eating
Philosopher 1 finished eating
Philosopher 2 is eating
Philosopher 0 is eating
Philosopher 2 finished eating
Philosopher 4 is eating
Philosopher 0 finished eating
Philosopher 4 finished eating
chanchal-manwatkar@chanchal-manwatkar-Aspire-A515-56G:~$
```